

5. 합성곱 신경망 I

5.1 합성곱 신경망

4장에서 배운 딥러닝의 역전파에서는 순전파 과정에서 계산된 오차 정보가 신경망의 모든 노드를 거쳐 전달함

-> 이러한 계산 과정은 복잡하고 많은 자원(CPU 혹은 GPU, 메모리)을 요구, 계산하는 데도 오래 걸림

-> 합성곱 신경망은 이미지 전체를 한 번에 계산하는 것이 아니라 **이미지의 국소적 부분을 계산**, 시간과 자원을 절약하면서 이미지의 세밀한 부분까지 분석할 수 있음

5.1.1 합성곱층의 필요성

- 합성곱 신경망은 이미지나 영상을 처리하는 데 유용함

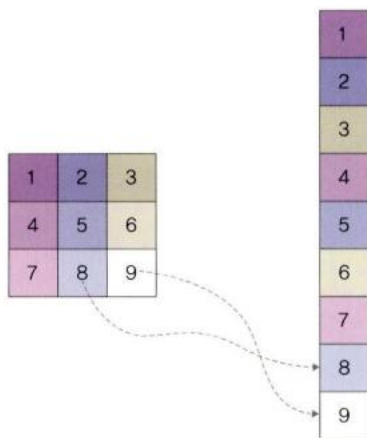
예를 들어 3×3 흑백(그레이스케일) 이미지를 분석한다고 가정

이미지 분석에서는 3×3 배열을 오른쪽과 같이 펼쳐서(flattening) 각 픽셀에 가중치를 곱하여 은닉층으로 전달하게 됨

하지만 이미지를 펼쳐서 분석하면 **데이터의 공간적 구조를 무시하게 됨**

→ 이것을 방지하기 위해 도입된 것이 **합성곱층**

♥ 그림 5-1 합성곱층 원리



5.1.2 합성곱 신경망 구조

합성곱 신경망(Convolutional Neural Network, CNN 또는 ConvNet)

- 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망
- 다차원 배열 데이터를 처리하도록 구성
- 컬러 이미지 같은 다차원 배열 처리에 특화

합성곱 신경망은 다음 다섯 개 계층으로 구성

1. 입력층

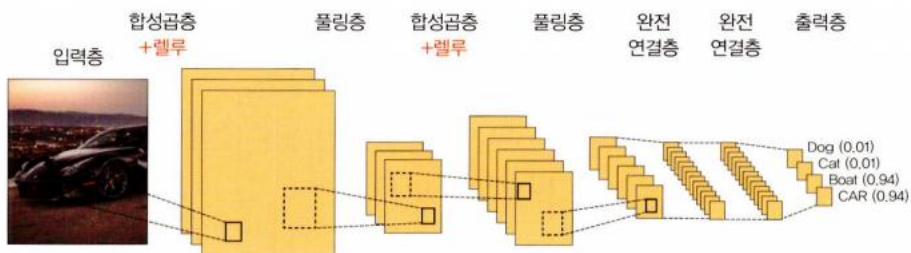
2. 합성곱층
3. 풀링층
4. 완전연결층
5. 출력층

합성곱 신경망은 **합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터(feature vector)를 추출**

→ 추출된 주요 특성 벡터들은 완전연결층을 거치면서 1차원 벡터로 변환

→ 출력층에서 활성화 함수인 **소프트맥스(softmax)** 함수를 사용하여 최종 결과 출력

▼ 그림 5-2 합성곱 신경망 구조



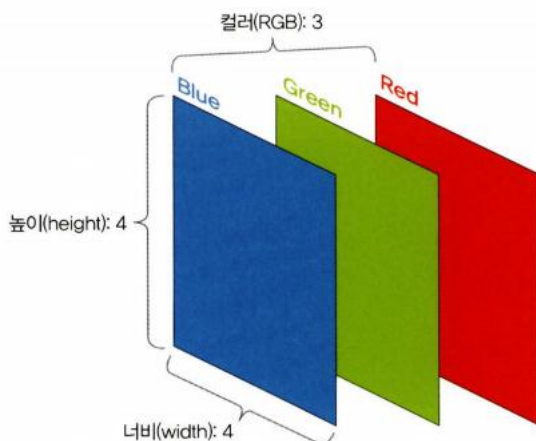
입력층(input layer)

- 입력 이미지 데이터가 최초로 거치게 되는 계층
- 이미지는 단순 1차원 데이터가 아닌 **높이(height), 너비(width), 채널(channel)** 값을 갖는 3차원 데이터

채널 : 그레이스케일(gray scale) : 1, 컬러(RGB) : 3

예) 높이 4, 너비 4, 채널 RGB 3 → 이미지 형태(shape) = (4, 4, 3)

▼ 그림 5-3 채널



합성곱층(convolutional layer) : 입력 데이터에서 특성을 추출하는 역할

이미지에 대한 특성을 감지하기 위해 **커널(kernel)** 또는 **필터(filter)** 사용

- 커널/필터가 이미지의 모든 영역을 훑으면서 특성을 추출
- 추출된 결과물을 **특성 맵(feature map)**이라고 함
- 일반적으로 커널은 3×3, 5×5 크기로 적용
- **스트라이드(stride)** : 지정된 간격에 따라 순차적으로 이동

합성곱 연산 과정

입력 이미지에 3×3 필터를 적용하고, 같은 위치의 값끼리 곱한 후 모두 더함 → 계산 결과가 출력의 한 값이 됨

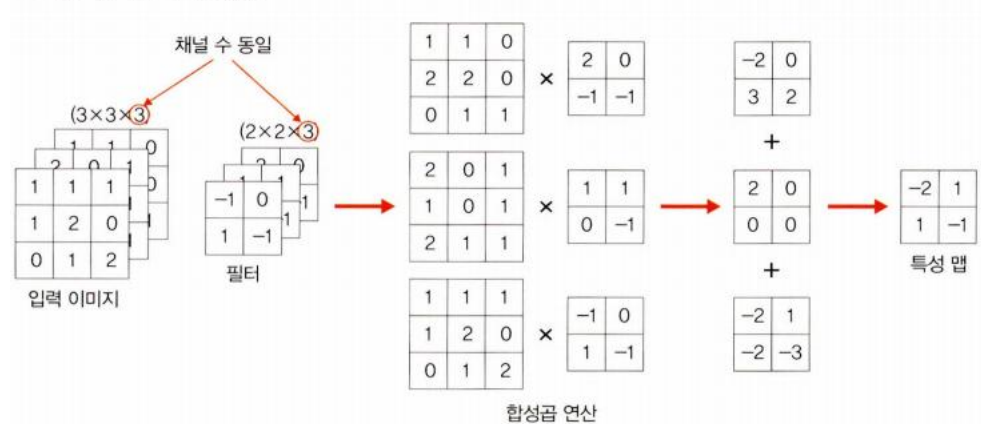
스트라이드가 1이므로 필터를 한 칸씩 이동시키며 같은 계산을 반복 → 필터가 입력 이미지 전체를 이동하면 하나의 특성 맵이 생성됨

컬러 이미지의 합성곱

컬러 이미지는 서로 다른 방법을 사용

- 빨간 채널이 3, RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해 출력값을 얻는 방법
- 각 색의 스칼라 곱 및 덧셈 방법은 동일
- 채널 개수를 세 개라고 오해하기 쉬운데 실제로는 필터 개수가 한 개라는 점에 주의

▼ 그림 5-10 컬러 이미지 합성곱

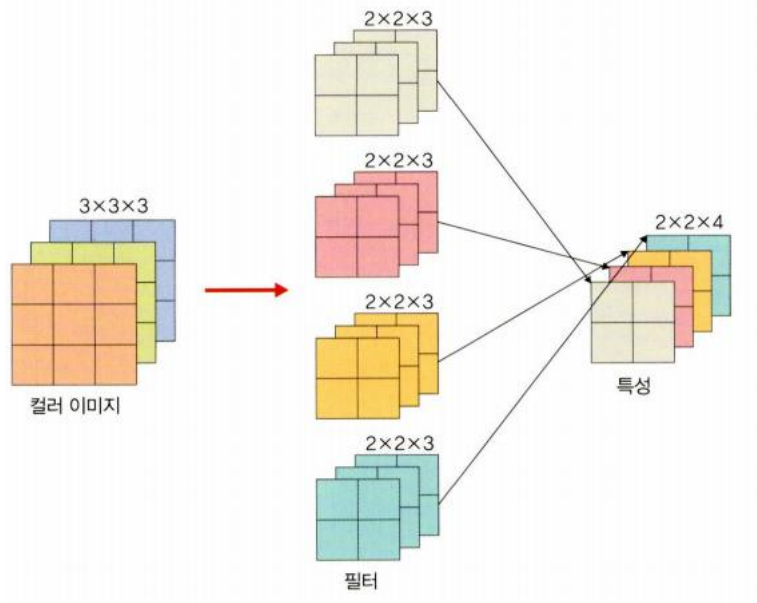


필터가 두 개 이상인 합성곱

그림 5-11처럼 필터가 두 개 이상인 경우

- 각각의 필터에서 합성곱 수행
- 필터마다 결과 생성
- 필터가 네 개라면 출력도 네 개 생성

▼ 그림 5-11 필터가 2 이상인 합성곱



합성곱층 출력 크기

- 입력 데이터

$W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)

- 하이퍼파라미터

- 필터 개수 : K
- 필터 크기 : F
- 스트라이드 : S
- 패딩 : P

- 출력 데이터

$$W_2 = (W_1 - F + 2P) / S + 1$$

$$H_2 = (H_1 - F + 2P) / S + 1$$

$$D_2 = K$$

풀링층(pooling layer)

- 합성곱층과 유사하게 특성 맵의 차원을 **다운 샘플링** → 연산량 감소
- 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있도록 함

- 다운 샘플링(sub-sampling) : 이미지를 축소하는 것

Note ≡ 다운 샘플링

다운 샘플링(sub-sampling)은 다음 그림과 같이 이미지를 축소하는 것입니다.

♥ 그림 5-12 다운 샘플링



풀링 연산에는 두 가지가 사용

- 최대 풀링(max pooling) : 대상 영역에서 최댓값을 추출
- 평균 풀링(average pooling) : 대상 영역에서 평균을 반환

대부분의 합성곱 신경망에서는 최대 풀링 사용

-> 평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문

최대 풀링

♥ 그림 5-13 첫 번째 최대 풀링 과정



두 번째 최대 풀링 과정

12, -1, 0, 1 값 중에서 최댓값(12)을 선택합니다.

♥ 그림 5-14 두 번째 최대 풀링 과정



세 번째 최대 풀링 과정

2, -3, 3, -2 값 중에서 최댓값(3)을 선택합니다.

♥ 그림 5-15 세 번째 최대 풀링 과정



네 번째 최대 풀링 과정

0, 1, 4, -1 값 중에서 최댓값(4)을 선택합니다.

♥ 그림 5-16 네 번째 최대 풀링 과정



평균 풀링

평균 풀링 계산도 최대 풀링과 유사하게 진행

다만 최댓값 대신 평균값 계산

교재 예)

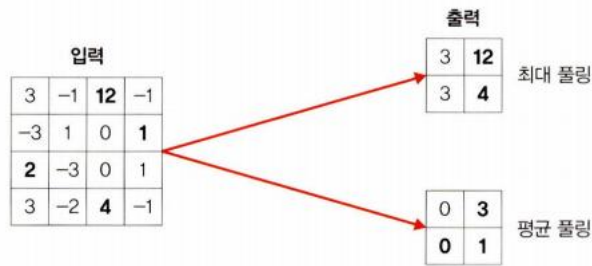
$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

♥ 그림 5-17 최대 풀링과 평균 풀링 비교



풀링층 출력 크기

입력 데이터 : $W_1 \times H_1 \times D_1$

하이퍼파라미터

- 필터 크기 : F
- 스트라이드 : S

출력 데이터

$$W_2 = (W_1 - F) / S + 1$$

$$H_2 = (H_1 - F) / S + 1$$

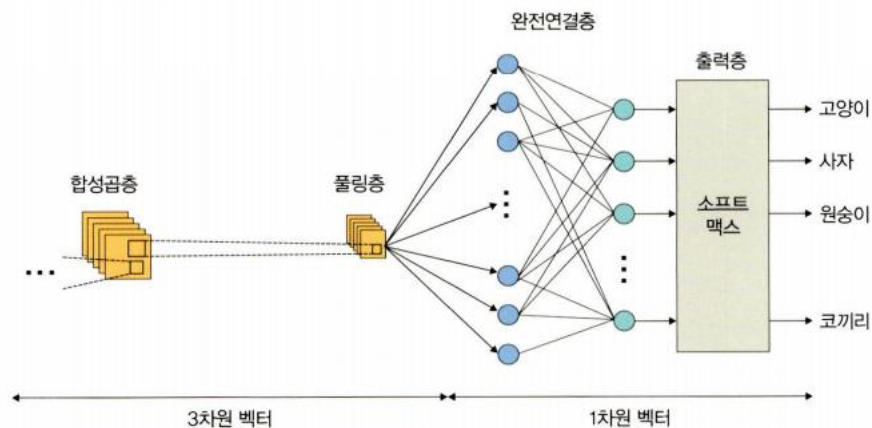
$$D_2 = D_1$$

완전연결층

합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 **완전연결층(fully connected layer)**으로 전달

이 과정에서 이미지는 **3차원 벡터** → **1차원 벡터**로 펼쳐지게(flatten) 됨

♥ 그림 5-18 완전연결층



출력층(output layer)

- 소프트맥스 활성화 함수 사용

- 입력받은 값을 0~1 사이의 값으로 출력
- 이미지가 각 레이블(label)에 속할 확률 값이 출력
- 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류

1D 합성곱

- 필터가 시간을 축으로 좌우로만 이동
- 입력(W)과 필터(k)에 대한 출력은 W'
- 출력 형태는 1D 배열

교재 예)

입력

[1, 1, 1, 1, 1]

필터

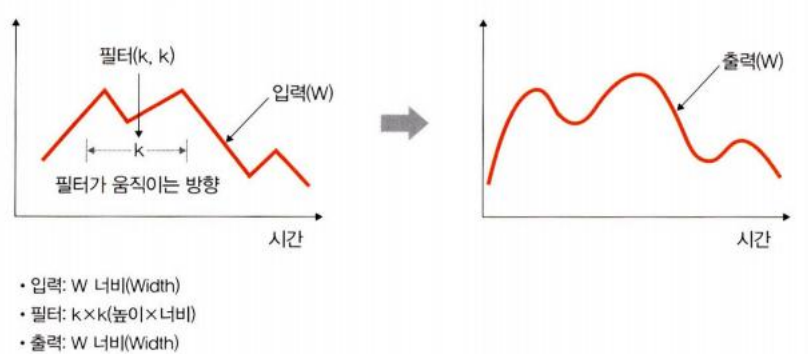
[0.25, 0.5, 0.25]

출력

[1, 1, 1]

→ 그래프 곡선을 완화할 때 많이 사용

▼ 그림 5-19 1D 합성곱



2D 합성곱

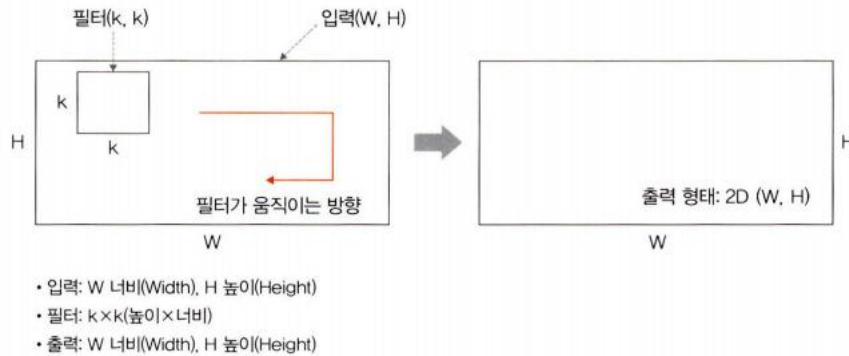
- 필터가 방향 두 개로 움직이는 형태

입력 : (W, H) / 필터 : (k, k) / 출력 : (W' , H') → 출력 형태는 2D 행렬

2D 합성곱

2D 합성곱은 필터가 다음 그림과 같이 방향 두 개로 움직이는 형태입니다. 즉, 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)가 되며, 출력 형태는 2D 행렬이 됩니다.

♥ 그림 5-20 2D 합성곱



3D 합성곱

- 필터가 움직이는 방향이 세 개

입력 : (W, H, L)

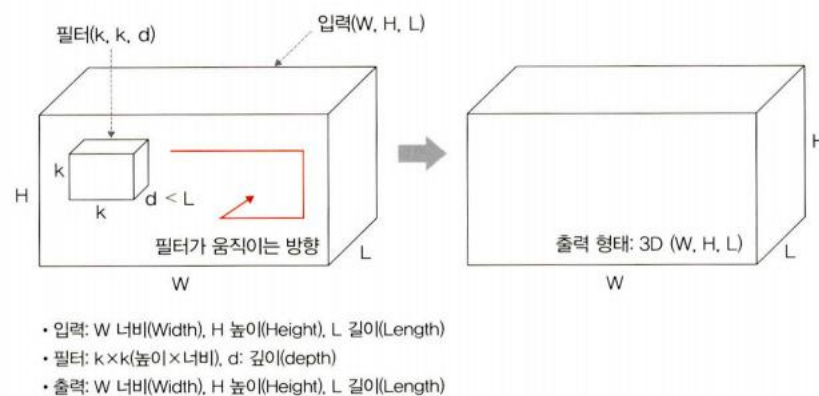
필터 : (k, k, d)

출력 : (W', H', L')

→ 출력은 3D 형태

→ 이때 $d < L$ 을 유지하는 것이 중요

♥ 그림 5-21 3D 합성곱



3D 입력을 갖는 2D 합성곱

입력이 224×224×3, 112×112×32 같은 3D 입력인데 출력 형태가 3D가 아닌 2D 형태인 경우

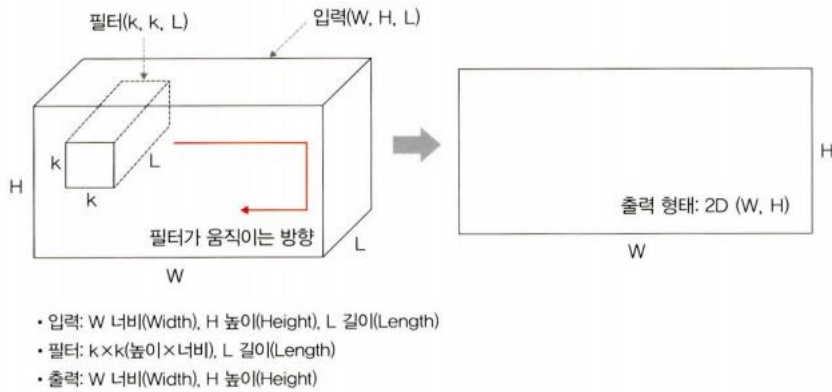
입력 : (W, H, L) / 필터 : (k, k, L) / 출력 : (W', H')

필터가 움직이는 방향은 두 개이기 때문에 2D 합성곱

대표적 사례

- LeNet-5
- VGG

♥ 그림 5-22 3D 입력을 갖는 2D 합성곱



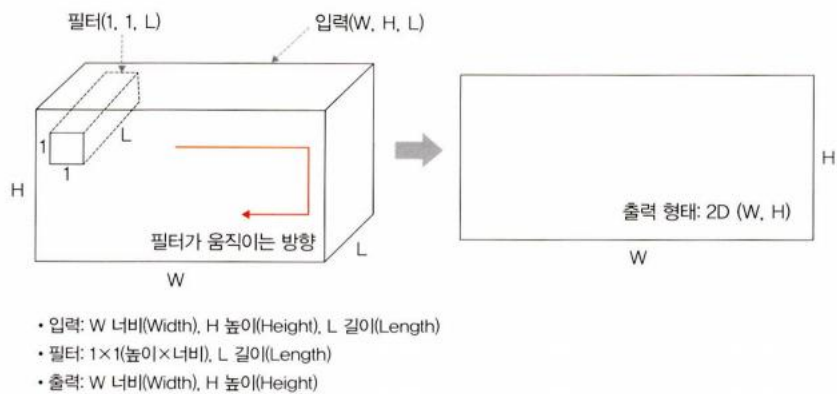
1×1 합성곱

- 3D 형태로 입력

입력 : (W, H, L)/필터 : (1, 1, L)/출력 : (W, H)

- 1×1 합성곱에서 채널 수를 조정하면 연산량 감소 효과
- 대표적 사례 : GoogLeNet

♥ 그림 5-23 1×1 합성곱



5.2 합성곱 신경망 맛보기

fashion_mnist 데이터셋을 사용하여 합성곱 신경망 직접 구현

fashion_mnist 데이터셋

fashion_mnist 데이터셋

- 토치비전(torchvision)에 내장된 예제 데이터
- 운동화, 셔츠, 샌들 같은 작은 이미지 모음
- 기존 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 **28×28 픽셀 이미지 7만 개**로 구성
- 데이터셋을 자세히 살펴보면 훈련 데이터(train_images)는 0~255 사이 값을 갖는 28×28 크기의 넘파이(NumPy) 배열
- 레이블(정답) 데이터(train_labels)는 0~9까지 정수 값을 갖는 배열

라이브러리 호출

예제 진행을 위해 필요한 라이브러리 호출

- NumPy : 난수 생성 및 배열 생성·조작
- matplotlib.pyplot : 이미지 및 데이터 시각화
- torch : CPU/GPU 설정 및 딥러닝 연산 수행
- torch.nn : 신경망을 구성하는 계층 및 모듈 제공
- Variable : 역전파에 필요한 미분 값 계산에 사용
- torch.nn.functional : 신경망 연산을 함수 형태로 사용
- torchvision : 이미지 관련 데이터셋 제공
- torchvision.transforms : 데이터 전처리 및 이미지를 텐서로 변환
- Dataset : 데이터를 데이터셋 형태로 다루기 위한 클래스
- DataLoader : 데이터를 배치 단위로 불러오고 순서를 섞어 사용

CPU 혹은 GPU 장치 확인

파이토치는 기본적으로 GPU 사용을 권장

하지만 GPU가 장착되지 않은 환경에서도 정상적으로 실행하고 사용할 수 있음

```
* device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

GPU 사용

GPU 하나를 사용할 때

- torch.device
- model.to(device)

사용

다수의 GPU를 사용한다면 nn.DataParallel 이용

- 배치 크기(batch size)가 알아서 각 GPU로 분배되는 방식

- GPU 수만큼 배치 크기도 늘려 주어야 함

fashion_mnist 데이터셋 내려받기

torchvision.datasets.FashionMNIST() 사용

주요 파라미터

① 첫 번째 파라미터

- FashionMNIST를 내려받을 위치 지정

② download

- True로 변경
- 첫 번째 파라미터 위치에 해당 데이터셋이 있는지 확인한 후 내려받음

③ transform

- 이미지를 텐서(0~1)로 변경

DataLoader

내려받은 fashion_mnist 데이터를 메모리로 불러오기 위해 **데이터로더(DataLoader)**에 전달

torch.utils.data.DataLoader()

- 원하는 크기의 배치 단위로 데이터 불러오기
- 순서가 무작위로 섞이도록(shuffle) 할 수 있음

주요 파라미터

① 첫 번째 파라미터

- 데이터를 불러올 데이터셋 지정

② batch_size

- 데이터를 배치로 묶음
- 교재에서는 batch_size=100
- 100개 단위로 데이터를 묶어서 불러옴

np.random

np.random

- 무작위로 데이터를 생성할 때 사용

np.random.randint()

- 이산형 분포를 갖는 데이터에서 무작위 표본 추출

예)

`np.random.randint(10)`

→ 0~9의 임의의 숫자

`np.random.randint(1, 10)`

→ 1~9의 임의의 숫자

`np.random.rand(8)`

→ 0~1 사이 난수를 (1×8) 형태로 출력

`np.random.rand(4, 2)`

→ 난수를 4×2 형태로 출력

`np.random.randn()`

- 평균이 0이고 표준편차가 1인 가우시안 정규분포

`np.random.randn(8)`

→ (1×8)

`np.random.randn(4, 2)`

→ 4×2

심층 신경망 모델 생성

합성곱 신경망과 합성곱 신경망이 아닌 심층 신경망의 비교를 위해 먼저 심층 신경망 생성 후 학습

즉, ConvNet이 적용되지 않은 네트워크를 먼저 생성

FashionDNN(nn.Module) 사용

클래스(class) / 객체(object)

클래스 형태 모델은 항상 `torch.nn.Module`을 상속

- `__init__()`

- 객체가 갖는 속성 값을 초기화
- 객체가 생성될 때 자동으로 호출

- `super(FashionDNN, self).__init__()`

- FashionDNN이 부모(super) 클래스를 상속받는다라는 의미

- 객체(object)

- 메모리를 할당받아 프로그램에서 사용되는 모든 데이터
- 변수, 함수 등도 모두 객체라고 할 수 있음

객체 사용 방식 : 객체명 = 클래스명()

클래스와 함수

함수(function)

- 특정 작업만 수행하도록 독립적으로 설계된 프로그램 코드
- 함수를 호출하면 특정 작업을 수행하고 결과값 반환

클래스(class)

- 함수뿐만 아니라 관련된 변수까지 한꺼번에 묶어서 관리하고 재사용할 수 있도록 함
- 여러 객체를 독립적으로 사용할 수 있음

nn / Linear

nn : 딥러닝 모델(네트워크) 구성에 필요한 모듈이 모여 있는 패키지

Linear : 단순 선형 회귀 모델을 만들 때 사용

Ex. `nn.Linear(in_features=784, out_features=256)`

`in_features` : 입력의 크기(input size), `out_features` : 출력의 크기(output size)

Dropout

`torch.nn.Dropout(p)`

- `p`만큼의 비율로 텐서의 값이 0이 됨
- 0이 되지 않는 값들은 기존 값에 일정 값을 곱해 커짐

교재 예) `p=0.3` → 전체 값 중 0.3의 확률로 0이 됨

forward()

- 모델이 학습 데이터를 입력받아 순전파(forward propagation) 학습 진행
- 반드시 `forward`라는 이름의 함수여야 함
- 모델이 학습 데이터를 입력받아 순전파 연산을 진행하는 함수
- 객체를 데이터와 함께 호출하면 자동으로 실행

view()

파이토치의 `view`

- 넘파이의 `reshape`와 같은 역할
- 텐서의 크기(shape)를 변경

`input_data.view(-1, 784)` → `input_data`를 (?, 784) 크기로 변경

- 첫 번째 차원 -1 : 파이토치에 맡김

- 두 번째 차원 : 길이가 784가 되도록 함

활성화 함수 지정

두 가지 방법

- `F.relu()` : `forward()` 함수에서 정의
- `nn.ReLU()` : `__init__()` 함수에서 정의

nn.xx와 nn.functional.xx

nn.xx

- `nn.Conv2d` : 클래스
- `nn.Module` 클래스를 상속받아 사용
- 먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달
- `nn.Sequential` 내에 위치
- 파라미터를 새로 정의할 필요 없음

nn.functional.xx

- `nn.functional.conv2d` : 함수
- `def function(input)`으로 정의된 순수한 함수
- 함수를 호출할 때 하이퍼파라미터와 데이터 전달
- `nn.Sequential`에 위치할 수 없음
- 가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

▼ 표 5-1 nn.xx와 nn.functional.xx의 사용 방법 비교

구분	nn.xx	nn.functional.xx
형태	<code>nn.Conv2d</code> : 클래스 <code>nn.Module</code> 클래스를 상속받아 사용	<code>nn.functional.conv2d</code> : 함수 <code>def function (input)</code> 으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	<code>nn.Sequential</code> 내에 위치	<code>nn.Sequential</code> 에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

심층 신경망에 필요한 파라미터 정의

모델을 학습시키기 전에 손실 함수, 학습률(learning rate), 옵티마이저(optimizer) 정의

심층 신경망 모델 학습

num_epochs = 5

학습 코드에서 사용되는 내용

- 리스트(list)를 비어 있는 배열로 생성
- append 메서드로 데이터 하나씩 추가
- for 구문을 사용하여 레코드(행, 가로줄)를 하나씩 가져옴

모델이 데이터를 처리하기 위해서는 **모델과 데이터가 동일한 장치(CPU 또는 GPU)에 있어야 함**

- model.to(device)
- images.to(device)
- labels.to(device)

CPU에서 처리된 데이터를 GPU 모델에 적용하거나 그 반대의 경우 런타임 오류 발생

Autograd / Variable

Autograd

- 자동 미분을 수행하는 파이토치의 핵심 패키지
- 자동 미분에 대한 값을 저장하기 위해 테이프(tape) 사용
- 순전파 단계에서 데이터에 수행되는 모든 연산을 저장
- 역전파(backward) 단계에서 저장된 값을 사용

Variable

- 역전파를 위한 미분 값을 자동으로 계산할 수 있도록 제공

정확도

분류 문제에 대한 정확도 : $\text{classification accuracy} = \text{correct predictions} / \text{total predictions}$

백분율 : $\text{classification accuracy} = \text{correct predictions} / \text{total predictions} \times 100$

오분류율 또는 오류율 : $\text{error rate} = (1 - \text{correct predictions} / \text{total predictions}) \times 100$

정확도 사용 시 주의

클래스가 세 개 이상일 때 주의

- 정확도가 80% 이상이어도 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지 알 수 없음
- 정확도가 90% 이상이어도 100개의 데이터 중 90개가 하나의 클래스에 속한다면, 모든 데이터를 특정 클래스라고 예측해도 90%의 결과가 나올 수 있음

- 데이터 특성에 따라 정확도를 잘 관측해야 함

심층 신경망 학습 결과

모델 훈련 결과

- 반복이 진행되면서 정확도 증가
- 최종적으로 정확도 **86%** 정도

→ 심층 신경망 모델의 생성과 성능 평가 후 합성곱 신경망 생성

합성곱 네트워크 생성

FashionCNN(nn.Module) 사용

주요 구성 :

nn.Sequential

- `__init__()`에서 사용할 네트워크 모델 정의
- `forward()` 함수에서 구현될 순전파를 계층(layer) 형태로 작성
- 계층을 차례로 쌓을 수 있도록 수식과 활성화 함수를 연결
- 데이터가 각 계층을 순차적으로 지나갈 때 사용
- 여러 개의 계층을 하나의 컨테이너에 구현하는 방법

Conv2d

합성곱층(conv layer)

- 합성곱 연산을 통해 이미지의 특징을 추출
- 커널(또는 필터)이라는 $n \times m$ 크기의 행렬이 높이 $\times$ 너비 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값을 곱한 후 모두 더한 값을 출력

- `nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)`

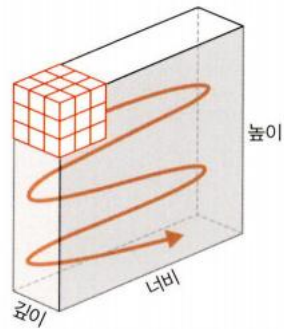
- `in_channels`

- 입력 채널의 수
- 흑백 이미지 : 1
- RGB 이미지는 3인 경우가 많음

- 채널

- 컬러 이미지는 일반적으로 Red, Green, Blue 세 개 채널
- 3차원으로 생각하면 채널은 깊이(depth)를 의미

♥ 그림 5-27 채널



out_channels

- 출력 채널의 수

- kernel_size

- 커널 크기
- 필터라고도 함
- 이미지 특징을 찾아내기 위한 공용 파라미터
- CNN에서 학습 대상은 필터 파라미터
- 일반적으로 3×3, 5×5 사용

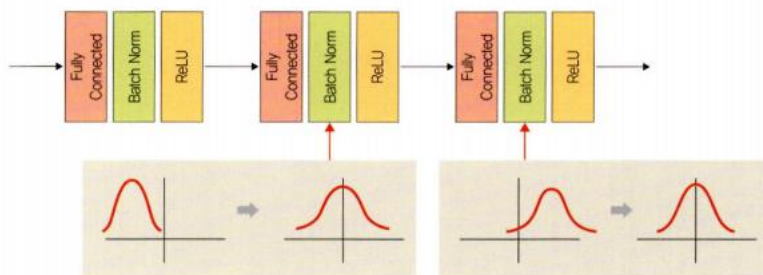
- padding

- 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채움
- 패딩 값이 클수록 출력 크기도 커짐

BatchNorm2d

- 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 **평균과 분산을 이용해 정규화**
- 배치 단위나 계층에 따라 입력값 분포가 달라도 정규화를 통해 분포를 가우시안 형태로 만듦
- 평균 : 0
- 표준편차 : 1

♥ 그림 5-28 BatchNorm2d



MaxPool2d

- 이미지 크기를 축소시키는 용도로 사용

- 합성곱층의 출력 데이터를 입력으로 받아 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조

풀링 방법

- 최대 풀링(max pooling)
- 평균 풀링(average pooling)
- 최소 풀링(min pooling)

- kernel_size

- $m \times n$ 형태로 구성된 가중치

- stride

- 입력 데이터에 커널(필터)을 적용할 때 이동할 간격
- stride 값이 커지면 출력 크기는 작아짐

완전연결층

클래스를 분류하기 위해 이미지 형태의 데이터를 배열 형태로 변환

Conv2d의 하이퍼파라미터 값에 따라 출력 크기가 달라짐

줄어든 출력 크기는 최종적으로 분류를 담당하는 **완전연결층(fully connected layer)**으로 전달

`nn.Linear(in_features=64*6*6, out_features=600)`

in_features : 입력 데이터의 크기

out_features : 출력 데이터의 크기

Conv2d 계층 출력 크기

출력 크기 = $(W - F + 2P) / S + 1$

- W : 입력 데이터 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층 출력 크기

출력 크기 = IF / F

- IF : 입력 필터의 크기(input_filter_size), 또는 바로 앞 Conv2d의 출력 크기
- F : 커널 크기(kernel_size)

out.view()

합성곱층에서 완전연결층으로 변경되므로 데이터 형태를 1차원으로 변경

```
out = out.view(out.size(0), -1)
```

out.size(0) → 100, 따라서 (100, ?) 크기의 텐서로 변경

-1 → 행(row)의 수는 정확히 알고 있지만 열(column)의 수를 알지 못할 때 사용

심층 신경망과 합성곱 신경망 비교

교재에서는 합성곱 신경망의 정확도가 **심층 신경망과 비교하여 약간 높다고** 설명

- 심층 신경망과 별 차이가 없기 때문에 좀 더 간편한 심층 신경망만 사용해도 무난할 것 같지만
- 실제로 이미지 데이터가 많아지면 단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능
- 따라서 **합성곱 신경망을 생성할 수 있도록 학습해야 함**